rsedit — Rapporto sui problemi del codice

Le schede dell'analisi della revisione `08a853d`, verificate sulla revisione `3d846f1`

Indice

1 Che cosa contiene questo rapporto 2
1.1 Le schede dell'interprete 2
1.2 Come leggere una scheda 2
1.3 Come è stato condotto l'esame 2
1.4 Riepilogo 3
2 Buffer, modifiche e annullamento 5
3 Tastiera, comandi e primitive dell'editor 6
4 Minibuffer 7
5 File e salvataggio 8
6 Avvio, configurazione e registro 9
7 Lavoro in background 10
8 Modi, sintassi e colorazione 11
9 Ricerca, sostituzione e liste di risultati 12
10 Comandi di shell 13
11 Disegno e terminale 14
12 La finestra grafica 15
13 Documentazione, commenti, codice morto, build e test 16
14 Appendici 17
14.1 Le schede risolte 17
14.2 Differenze da Emacs che non sono difetti 19
14.3 Parti esaminate e trovate solide 19

1 Che cosa contiene questo rapporto

Questo documento elenca i problemi trovati analizzando il codice di rsedit. È separato di proposito dal manuale di architettura, `architettura-editor.typ`. Il manuale descrive come il codice funziona e resta valido anche dopo una correzione; questo rapporto descrive dove il codice non fa ciò che dovrebbe. Quando un problema viene risolto, la sua scheda passa nell'appendice delle schede risolte, con il commit che lo ha risolto.

Ogni problema ha un identificativo stabile, formato dalla sigla dell'area e da un numero (`BUF-2`, `CMD-1`…), che si può citare in un commit o in una discussione; nel codice lo si cita solo in un `TODO(ID)`, e solo finché la scheda è aperta. Le schede sono generate da un'unica tabella di dati all'inizio del sorgente: per aggiungerne una, o per segnarne una come risolta, basta modificare quella tabella, e il riepilogo si aggiorna da solo.

1.1 Le schede dell'interprete

Le schede delle aree LET (il lettore) e INT (il valutatore e la libreria di base) riguardano l'interprete, che da `6d30567` è il crate esterno `risp`. Sono passate nel rapporto del suo repository, `doc/rapporto-problemi.typ`, con gli stessi identificativi. Le schede di qui che vi rimandano lo dicono nel testo, per esempio «INT-2, nel rapporto di risp». rsedit usa la revisione di risp fissata nel suo `Cargo.lock`, oggi `b2d000b`: una scheda che risp risolve dopo quella revisione resta un difetto di rsedit finché la dipendenza non si aggiorna.

1.2 Come leggere una scheda

Ogni scheda riporta:

- **Gravità**, secondo la scala qui sotto.
- **Dove**: i file e le funzioni, o i tipi, in cui sta il difetto. I percorsi senza prefisso sono relativi a `core/src/`; quelli che cominciano con `src/` riguardano la TUI e la GUI, quelli con `core/lisp/` i moduli Lisp.
- **Verifica**: «verificato» se il comportamento è stato riprodotto eseguendo il codice (il testo dice come, e su quale revisione), «dal codice» se la conclusione viene dalla sola lettura.
- **Cosa succede**, il meccanismo del difetto; **Come riprodurlo**, quando serve; **Effetto**, ciò che l'utente vede; **Correzione proposta**.

Tutte le schede dell'elenco sono aperte; le risolte sono nell'appendice.

Gravità	Significato
ALTA	L'editor termina, si blocca senza via d'uscita o perde dati; oppure un contenuto esterno può far eseguire azioni al terminale; oppure l'editor finisce in uno stato da cui si esce solo riavviando.
MEDIA	Un comportamento sbagliato o un rallentamento di secondi in un uso realistico, senza perdita di dati.
BASSA	Casi limite, incoerenze, codice morto, commenti e documentazione superati.

1.3 Come è stato condotto l'esame

Il codice della revisione `08a853d` è stato letto per intero: l'interprete (allora `core/src/lisp`, ora il crate `risp`), la libreria dell'editor (`core/src`), la TUI (`src/`) e i moduli Lisp (`core/lisp`). Ogni comportamento sospetto è stato riprodotto, dove possibile, con piccoli programmi esterni che usano `rsedit_core` come una libreria:

- un **esecutore di scenari** che valuta espressioni Lisp, preme tasti attraverso il percorso reale dei tasti, attende i lavori di background e stampa buffer, area messaggi ed evidenziazioni della fotografia dello schermo;

- un **fuzzer delle primitive**, che chiama ciascuna delle 628 funzioni registrate (tranne quelle che toccano file o processi esterni) con 807 combinazioni di argomenti anomali (zero, negativi, 10^{23}, infinito, NaN, stringhe vuote e non ASCII, liste puntate, vettori, mappe, lambda), una volta su un buffer vuoto e una su un buffer con testo multiriga, caratteri non ASCII, tab, una regione attiva e finestre divise.

I fuzzer del lettore e delle forme speciali, che hanno trovato le schede dell'interprete, sono descritti nel rapporto di risp. Le misure di tempo sono state prese con build di release su una macchina condivisa, quindi vanno lette come ordini di grandezza.

1.3.1 La verifica sulla revisione `3d846f1`

Ogni scheda è stata poi verificata sulla revisione `3d846f1`. Il codice che descrive è stato riletto, e la storia di git ha detto quale commit lo ha cambiato. Dove la scheda dice «verificato», la riproduzione è stata eseguita di nuovo, con lo stesso esecutore di scenari o con due editor veri in un terminale virtuale. Le misure di tempo e di memoria non sono state ripetute dove il codice misurato non è cambiato, e la scheda lo dice. Il campo **Dove** nomina ora file e funzioni, non righe.

Delle 51 schede di rsedit, 11 sono risolte. Sette lo sono in parte e restano aperte con il testo aggiornato a ciò che manca: BUF-1, CMD-1, CMD-3, CMD-6, MIN-3, FIL-5 e DOC-1. La verifica ne ha trovate quattro nuove: AVV-6, BKG-2, RIC-7 e DOC-5. Le schede trovate dopo, nei passi della documentazione che riscrivono il codice, dicono nella verifica dove sono state riprodotte, o che vengono dalla lettura del codice: RIC-8, BUF-4, CMD-7, CMD-8, BKG-3, SHL-5, MIN-4, RIC-9, MIN-5, FIL-6, SIN-7, SIN-8, SIN-9, AVV-7, AVV-8, GUI-1 e CMD-9. RIC-10 è stata trovata, con un test, mentre si correggeva la visita dei risultati per la directory di lavoro, e risolta nello stesso commit: è tra le risolte.

1.4 Riepilogo

> Sono aperti **4 problemi**: **0 di gravità alta**, 0 di gravità media e 4 di gravità bassa; 2 sono stati riprodotti eseguendo il codice. Altri 69 sono stati risolti dopo l'analisi, e sono nell'appendice.

Non restano problemi di gravità alta: dei quattro trovati, due riguardavano il terminale, che un file poteva comandare (TUI-1) e che un panic o un errore lasciavano in raw mode (TUI-2), e due facevano perdere lavoro senza dirlo, un salvataggio fallito (FIL-1) e due prompt aperti insieme (MIN-1). Le tre correzioni trasversali che l'analisi proponeva ci sono, nell'editor: nel WorkManager un panic ritira solo il lavoro che l'ha causato (BKG-1), una guardia ripristina il terminale (TUI-2), e le larghezze guidate dall'utente hanno un limite, mentre i conteggi si fermano al bordo del buffer (CMD-3). Il limite manca ancora nell'interprete (INT-2 e INT-13, nel rapporto di risp).

La tabella seguente elenca i problemi aperti, dal più grave; l'identificativo rimanda alla scheda.

ID	Problema	Gravità	Verifica
BUF-4	Sola lettura: kill, verbi, commenti e rettangoli tacciono; `kill-rectangle` conserva il blocco	BASSA	verificato
CMD-7	Windows: `insert-pasted-text` non converte CR e CRLF	BASSA	dal codice
CMD-9	vim: entrare e uscire da Insert costa quanto il file	BASSA	verificato
DOC-1	Commenti e docstring superati	BASSA	dal codice

Area	Descrizione	Alta	Media	Bassa	Risolte
BUF	Buffer, modifiche e annullamento	0	0	1	3
CMD	Tastiera, comandi e primitive dell'editor	0	0	2	7
MIN	Minibuffer	0	0	0	5
FIL	File e salvataggio	0	0	0	6
AVV	Avvio, configurazione e registro	0	0	0	8
BKG	Lavoro in background	0	0	0	3
SIN	Modi, sintassi e colorazione	0	0	0	9
RIC	Ricerca, sostituzione e liste di risultati	0	0	0	10
SHL	Comandi di shell	0	0	0	5
TUI	Disegno e terminale	0	0	0	8
GUI	La finestra grafica	0	0	0	1
DOC	Documentazione, commenti, codice morto, build e test	0	0	1	4

Tabella 1: I problemi aperti per area e gravità, e quelli risolti

2 Buffer, modifiche e annullamento

BUF-4 — In un buffer in sola lettura molti comandi non dicono nulla, e `kill-rectangle` conserva un blocco che non ha tolto

Gravità	BASSA
Dove	`primitives/killing.rs` (`cut_out`, e le kill che la usano), `primitives/verbs.rs` (`replace_span` e i verbi), `primitives/comments.rs` (`apply`), `primitives/rectangle.rs` (`cut`, `insert_at_edge`, `insert_block` e i comandi che li usano), `primitives/macros.rs` (`kmacro-insert-counter`)
Verifica	Riprodotto con un test temporaneo durante il passo 16 della documentazione, sul codice di `9fcd84b` con i passi 12–15, che non cambiano il comportamento. In un buffer in sola lettura `kill-line`, `kill-word`, `upcase-word`, `transpose-chars`, `join-line`, `comment-dwim` e `zap-to-char` non cambiano il testo e lasciano vuota l'area messaggi. `kill-rectangle` restituisce 3 e lascia il testo com'è, e dopo `(killed-rectangle)` restituisce le tre righe del blocco; `delete-rectangle`, `open-rectangle` e `string-rectangle` restituiscono 3 senza aver cambiato nulla; `kmacro-insert-counter` segnala l'errore «This buffer is read-only». Che il contatore sia già avanzato viene dalla lettura del codice.

Cosa succede. Le due porte rifiutano la modifica e restituiscono `false`, e dirlo all'utente tocca a chi le chiama, con `edited`, che scrive «Buffer is read-only». Lo fanno le primitive di `edits.rs`, `kill-region`, `yank`, `yank-pop` e `indent-line-to`. Le kill che passano da `cut_out` ne ricevono una stringa vuota e la danno all'anello, che la ignora; i verbi e i commenti rispondono nil; i comandi dei rettangoli scartano il risultato delle porte e rispondono con il numero di righe. `kill-rectangle` legge il blocco prima di cancellarlo, e lo conserva per `yank-rectangle` anche se la cancellazione è stata rifiutata. `kmacro-insert-counter` fa avanzare il contatore prima di inserire, e segnala un errore al posto del messaggio.

Effetto. (1) Un tasto che non fa nulla e non dice nulla: in una vista in sola lettura, come dired o l'aiuto, sembra un editor rotto e non un buffer protetto. (2) Dopo `kill-rectangle` in un buffer in sola lettura, `yank-rectangle` in un altro buffer inserisce un blocco che non è mai stato tolto, cioè lo duplica.

Correzione proposta. Far usare `edited` a tutti i comandi che modificano, a partire da `cut_out` e da `replace_span`; nei rettangoli fermarsi al primo rifiuto, senza conservare il blocco né spostare il punto; in `kmacro-insert-counter` inserire prima di far avanzare il contatore.

3 Tastiera, comandi e primitive dell'editor

CMD-7 — Su Windows un incolla tiene i ritorni a capo CR, e lì quattro test dell'incolla fallirebbero

> **Gravità** BASSA
> **Dove** `primitives/edits.rs` (la variante per Windows di `normalize_line_endings`, che usa `insert-pasted-text`), `tests/paste_tests.rs`
> **Verifica** Dalla lettura del codice: i test non girano su Windows, e la verifica dei passi compila soltanto per quel sistema.

Cosa succede. `insert-pasted-text` converte in LF ogni CRLF e ogni CR isolato prima di inserire, perché l'indice delle righe conta solo `\n`. Su Windows invece `normalize_line_endings` restituisce il testo com'è, e nessun commento dice perché. La GUI passa a `handle_paste` gli appunti del sistema, che su Windows hanno di solito i fine riga CRLF. `paste_tests` non esclude Windows, e lì `a_paste_with_crlf_endings_becomes_lines`, `a_paste_with_bare_carriage_returns_becomes_lines`, `a_mix_of_endings_all_become_lines` e `a_trailing_carriage_return_opens_a_line` fallirebbero.

Effetto. Su Windows un testo di più righe incollato lascia un CR alla fine di ogni riga, e un testo con CR isolati diventa una riga sola.

Correzione proposta. Decidere che cosa vuole Windows: convertire come sugli altri sistemi, togliendo la variante, oppure scriverne il motivo e limitare i quattro test agli altri sistemi.

CMD-9 — In vim ogni sessione di Insert copia e confronta il buffer intero

> **Gravità** BASSA
> **Dove** `core/lisp/features/vim.lisp` (`vim--begin-insert`, che ricorda `(buffer-string)`, e `vim--finish-insert`, che lo confronta con `string-diff`)
> **Verifica** Misurato con un test temporaneo durante il passo 21 della documentazione, compilato in release: su un buffer di 100 KB `i`, `x` ed Esc costano 1,3 ms, su uno di 2 MB 47 ms, mentre un movimento costa 0,2 ms in tutti e due.

Cosa succede. Per sapere che cosa la sessione ha cambiato, e farlo ripetere a `.`, `vim--begin-insert` copia il testo del buffer intero in una proprietà del buffer, e `vim--finish-insert` ne fa un'altra copia e confronta le due con `string-diff`. Il costo è proporzionale al file, non a ciò che si è scritto, e lo pagano ogni `i`, `a`, `o`, `c` e Replace, anche quando la sessione non cambia nulla; la copia resta in memoria per tutta la sessione.

Effetto. In un file di qualche megabyte entrare e uscire da Insert ha un ritardo visibile, che cresce con il file: decine di millisecondi a 2 MB, circa duecento a 8 MB. Sui file piccoli non si nota.

Correzione proposta. Ricordare l'intervallo toccato, dal punto d'ingresso e dalle modifiche che le porte registrano (l'undo le conosce già), invece del testo intero; oppure confrontare solo le righe tra la prima e l'ultima modificata.

4 Minibuffer

Nessuna scheda aperta: quelle dell'area sono tra le risolte.

5 File e salvataggio

Nessuna scheda aperta: quelle dell’area sono tra le risolte.

6 Avvio, configurazione e registro

Nessuna scheda aperta: quelle dell’area sono tra le risolte.

7 Lavoro in background

Nessuna scheda aperta: quelle dell’area sono tra le risolte.

8 Modi, sintassi e colorazione

Nessuna scheda aperta: quelle dell’area sono tra le risolte.

9 Ricerca, sostituzione e liste di risultati

Nessuna scheda aperta: quelle dell’area sono tra le risolte.

10 Comandi di shell

Nessuna scheda aperta: quelle dell’area sono tra le risolte.

11 Disegno e terminale

Nessuna scheda aperta: quelle dell’area sono tra le risolte.

12 La finestra grafica

Nessuna scheda aperta: quelle dell’area sono tra le risolte.

13 Documentazione, commenti, codice morto, build e test

DOC-1 — Commenti e docstring che descrivono un comportamento che non c'è più

> **Gravità** BASSA
> **Dove** vedi l'elenco
> **Verifica** Dalla lettura del codice.

Cosa succede.

- `primitives/shell.rs` (`shell-command-start`): il messaggio «The editor's background work has shut down» vale anche quando il sistema non dà un thread nuovo, l'altro caso in cui `WorkManager::spawn` rifiuta. Il commento accanto, che diceva lo stesso, è corretto dal passo 17 della documentazione.
- `primitives/occur.rs` (`grep--scan`): il messaggio «The editor's background work has shut down» vale anche quando il sistema non dà un thread nuovo, l'altro caso in cui `WorkManager::spawn` rifiuta. Il commento accanto, che diceva lo stesso, è corretto dal passo 18 della documentazione.
- `primitives/workers.rs` (`define-worker` e `background-call`, attraverso `start`): la riga del registro «the editor's work has shut down» vale anche quando l'esecutore non è partito, l'altro caso in cui `WorkManager::submit` rifiuta. Il commento accanto è corretto come quello di `shell.rs`.

La voce su `src/tui.rs` (`render_to`, il commento sull'area messaggi, che diceva il minibuffer agganciato alle ultime righe del frame) è corretta dal passo 20 della documentazione, e il commento è oggi in `src/tui/render.rs`. Le voci dell'analisi su `primitives/workers.rs` e su `shell-command-to-string` non valgono più: il codice e la docstring sono stati riscritti (`6de16a1`, `4f05134`). Le tre sui commenti della libreria di base dell'interprete sono nel rapporto di risp, dentro le schede INT-9, INT-16 e INT-21.

Correzione proposta. Aggiornarli insieme alle correzioni dei problemi a cui rimandano.

14 Appendici

14.1 Le schede risolte

Le schede risolte dopo l'analisi, con il commit che le ha risolte. Il testo completo di ciascuna sta nella storia di git di questo file; che cosa è cambiato, e perché, nel messaggio del commit.

ID	Problema	Gravità	Risolto in
BUF-1	`undo` e `redo` modificano il testo senza passare dalle due porte delle modifiche	MEDIA	`000b0a8`
BUF-2	`cursor_pos()` costa O(posizione): ogni fotogramma rilegge il buffer fino al punto	MEDIA	`64e33a3`
BUF-3	`GapBuffer::clone` è `todo!()` e `find_forward` va in underflow su un buffer vuoto	BASSA	`2ae6b6d`
CMD-1	I caratteri non ASCII non si possono digitare	MEDIA	`b9873db`
CMD-2	Mancano Canc, Inizio, Fine, PagSu, PagGiù e i tasti funzione	MEDIA	`3fb5e18`
CMD-3	Argomenti numerici enormi: allocazioni che terminano il processo e panic	MEDIA	`d86f0ea`
CMD-4	Le combinazioni di modificatori in `define-key` sono limitate e un errore va solo nel registro	BASSA	`02f67d9`
CMD-5	`self-insert` ignora l'argomento prefisso	BASSA	`02f67d9`
CMD-6	Un conteggio negativo non inverte i comandi che cancellano	BASSA	`02f67d9`
CMD-8	`close-floating-window` senza finestre flottanti sposta il focus sulla finestra 0, anche quando non esiste	BASSA	`02f67d9`
MIN-1	Due domande aperte insieme condividono `*Minibuffer*` e lasciano una finestra fantasma	ALTA	`b8829ad567399b388a0bc9dedf51`
MIN-2	`C-g` dentro il minibuffer non annulla la domanda	MEDIA	`041145b`
MIN-3	Il contenuto del minibuffer viene sostituito senza passare dalle porte delle modifiche, e solo fino al punto	BASSA	`5a89afd`
MIN-4	Un `minibuffer-read` con MODE nil apre il prompt nel modo `nil`, dove Invio ed Esc non rispondono	BASSA	`b14b96d`
MIN-5	`minibuffer-choose-completion` scrive nel buffer corrente, anche quando non è il prompt	MEDIA	`5a89afd`
FIL-1	Un salvataggio fallito con `C-x C-s` non viene segnalato	ALTA	`7d922071baf5fa4d98fbf1c02654`
FIL-2	I file vengono scritti sul posto: un errore a metà li lascia troncati	MEDIA	`fdf930d`
FIL-3	Un file non UTF-8 non si apre e non viene detto nulla	MEDIA	`de075c1`
FIL-4	Una modifica fatta durante il salvataggio può restare non salvata ma non segnata	BASSA	`848d312`
FIL-5	Il salvataggio automatico segna come salvato anche un tentativo fallito	BASSA	`848d312`
FIL-6	Con `auto-save-directory`, due percorsi diversi possono avere la stessa copia	BASSA	`848d312`
AVV-1	L'editor non parte se la directory corrente non è scrivibile	ALTA	`58d0b0d`
AVV-2	Gli errori di sintassi nei file Lisp caricati vengono ignorati in silenzio	MEDIA	`7433e5d`
AVV-3	Un errore di scrittura del registro manda l'editor in panic	MEDIA	`a20f050`
AVV-4	Il registro in memoria cresce senza limite	BASSA	`a20f050`
AVV-5	La configurazione predefinita viene scritta solo se manca	BASSA	`d86f0ea`
AVV-6	Due editor aperti insieme scrivono lo stesso `rsedit.log` e si cancellano le righe a vicenda	BASSA	`4ec00df`
AVV-7	Un errore in `init.lisp`, o in un modulo che richiede, impedisce all'editor di partire	MEDIA	`7433e5d`
AVV-8	Senza la directory della cache l'editor gira senza file di registro, e non lo dice	BASSA	`4ec00df`
BKG-1	Un panic in un lavoro di background uccide per sempre il thread condiviso	ALTA	`6de16a1`
BKG-2	Un lavoro dell'editor che va in panic sparisce per tutta la sessione, e lo dice solo il registro	BASSA	`c65efad`
BKG-3	`setenv` cambia l'ambiente del processo mentre il codice C di un altro thread può leggerlo	BASSA	`c65efad`

ID	Problema	Gravità	Risolto in
SIN-1	Una regione di sintassi che corrisponde alla stringa vuota blocca il thread di background	MEDIA	`173d8f5`
SIN-2	Colorare una riga molto lunga costa un tempo quadratico nella sua lunghezza	MEDIA	`173d8f5`
SIN-3	`write-file` che cambia il modo lascia i colori del modo precedente	BASSA	`cf0d8ac`
SIN-4	`make-mode` su un modo esistente lo sostituisce con uno vuoto	BASSA	`5aead86`
SIN-5	Il registro delle facce cresce per sempre e l'indice è a 16 bit	BASSA	`5aead86`
SIN-6	`forward-sexp` dall'interno di un simbolo salta alla fine dell'espressione successiva	MEDIA	`faa4694`
SIN-7	Una regola con GROUP colora tutta la corrispondenza quando il gruppo non vi prende parte	BASSA	`173d8f5`
SIN-8	`forward-sexp`, `kill-sexp` e `down-list` alla fine di una lista proseguono nelle liste che seguono	MEDIA	`faa4694`
SIN-9	`forward-sexp` si ferma alla fine di un commento a blocchi ma scavalca un commento di riga	BASSA	`faa4694`
RIC-1	Una sostituzione con un'espressione che può essere vuota mette tutte le sostituzioni nello stesso punto	MEDIA	`3f1df42`
RIC-2	La ricerca all'indietro con un'espressione regolare costa un tempo quadratico	MEDIA	`3f1df42`
RIC-3	Ogni sostituzione è un gruppo di annullamento a sé	BASSA	`3f1df42`
RIC-4	Limiti della ricerca confinata a una regione e delle scansioni su righe lunghe	BASSA	`3f1df42`
RIC-5	I risultati di occur, grep e compile puntano a righe che si spostano	BASSA	`01f27b2`
RIC-6	Alla fine di una ricerca incrementale il mark non è dove la ricerca è cominciata	BASSA	`1aa5cf7`
RIC-7	Una lista di `M-s o` e una di `M-s g` aprono due strisce, una sopra l'altra	BASSA	`01f27b2`
RIC-8	Un `query-replace` lasciato con un altro tasto resta aperto, con la corrispondenza evidenziata	BASSA	`01f27b2`
RIC-9	`results-put` e `results-select` prendono un BUFFER nil come il buffer chiamato `nil`	BASSA	`b14b96d`
RIC-10	Il segno di un risultato in un file resta quando la visita passa a un altro file	BASSA	`287a2eb`
SHL-1	Un comando che scrive molto su stderr si blocca	MEDIA	`02ea0e7`
SHL-2	Un solo byte non UTF-8 nell'output interrompe il comando	MEDIA	`02ea0e7`
SHL-3	Non c'è modo di fermare un comando in corso	BASSA	`02ea0e7`
SHL-4	Il nome di una pagina di manuale viene passato alla shell senza virgolette	BASSA	`a8e71ca`
SHL-5	Un comando di shell che ignora `TERM` sopravvive all'editor	BASSA	`a8e71ca`
TUI-1	I caratteri di controllo del testo arrivano al terminale: un file può comandarlo	ALTA	`543ef5f737128145564112ee107c0`
TUI-2	Un panic o un errore di I/O lasciano il terminale in raw mode	ALTA	`67a6a1d7a4e09d2854c57137dcaa5`
TUI-3	Le tabulazioni non vengono espanse	MEDIA	`efc7540`
TUI-4	I caratteri larghi contano come una colonna	MEDIA	`efc7540`
TUI-5	Un colore esadecimale con caratteri non ASCII manda l'editor in panic	MEDIA	`efc7540`
TUI-6	Le finestre flottanti non mostrano colori di sintassi né overlay	BASSA	`b5046e0`
TUI-7	`%p` nella riga di stato dice `All` solo per un buffer di una riga	BASSA	`b5046e0`
TUI-8	La TUI non usa lo schermo alternativo	BASSA	`b5046e0`
GUI-1	La finestra grafica si aggiorna ogni 250 ms anche quando nulla cambia	BASSA	`b5046e0`
DOC-2	Trappole latenti: `Debug` e `PartialEq` di `EditorState`	BASSA	`2ae6b6d`
DOC-3	Lo script di build presume dove si trova la directory `target`	BASSA	`f742701`
DOC-4	Un test di prestazioni instabile e clippy che fallisce sui test	BASSA	`10598fa`

ID	Problema	Gravità	Risolto in
DOC-5	Il codice delle liste di risultati lascia un metodo morto, due `if` da unire e righe da formattare	BASSA	`2ae6b6d`

14.2 Differenze da Emacs che non sono difetti

Alcuni comportamenti differiscono da Emacs per scelta, o comunque senza produrre risultati sbagliati. Sono elencati qui perché chi conosce Emacs non li scambi per errori. Quelle dell'interprete sono nel rapporto di risp.

- Le espressioni regolari usano la sintassi del crate `regex` di Rust, non quella di Emacs. `\(` è una parentesi letterale e i gruppi si scrivono `( )`. Non esistono lookaround né riferimenti all'indietro, ed è proprio ciò che garantisce un tempo lineare.
- Le liste di risultati, le ricerche e le sostituzioni non tornano mai all'inizio da sole: quando la lista finisce lo dicono, come scelta esplicita di progetto.

14.3 Parti esaminate e trovate solide

Nell'analisi queste parti sono state esercitate di proposito sui casi limite senza trovare problemi:

- lo scanner delle espressioni bilanciate (`modes/sexp.rs`): stringhe con escape alla fine del buffer, parentesi sbilanciate in entrambe le direzioni, letterali carattere, lifetime di Rust, stringhe raw, commenti a blocco annidati, commenti che contengono parentesi. Fuori da questi casi, più tardi, le verifiche hanno trovato SIN-6 e SIN-8
- la ricerca incrementale: parentesi letterali, espressioni non valide, cancellazione, ricerca fallita e ripresa dopo il fallimento;
- kill e yank, compreso `M-y`, e le macro di tastiera (con il limite alla ricorsione delle macro);
- `kill-buffer` di un buffer mostrato in due finestre, la chiusura dell'unica finestra, la chiusura di `*scratch*`;
- la sostituzione `a` → `aa`, che termina;
- i comandi di modifica ai bordi del buffer;
- `delete-file` sui link simbolici a directory, che cancella il link e non il contenuto della directory di destinazione;
- l'uscita con buffer non salvati: dopo un salvataggio fallito il buffer resta segnato come modificato e l'editor chiede conferma prima di uscire.